Authentication v/s Authorization

Contents

Authentication proves who you are. Authorization decides what you are allowed to do. Almost every security bug in a web application comes from mixing these two up or skipping one of them.

Generic questions

1. What is authentication?

Authentication (AuthN) verifies who you are. The caller claims an identity and the system checks that claim against evidence before trusting it.

The evidence is called a **factor**, and there are three kinds:

- **Something you know:** password, PIN.
- **Something you have:** phone OTP, hardware key, client certificate.
- **Something you are:** fingerprint, face.

MFA combines two kinds, so a stolen password alone is not enough. On success the caller gets a session or token; on failure the response is `401`.

> **Trap**
>
> `401 Unauthorized` is misnamed. It means **not authenticated**. A permission failure is `403 Forbidden`.

2. What is authorization?

Authorization (AuthZ) decides what an identity may do. It takes the identity, resource, action, and context, then returns allow or deny.

Authentication runs once; authorization runs on **every protected operation**. The rules come from a model:

- **RBAC (role-based):** permissions via roles. “Admins can delete.”
- **ABAC (attribute-based):** decision from attributes. “Managers approve expenses under 10k in their own department.”
- **ReBAC (relationship-based):** decision from a relationship graph, as in Google Zanzibar. “Edit if you own the doc or it was shared with you.”
- **ACLs and scopes:** a list attached to each resource, or OAuth scopes like `read:profile`.

A denial returns `403 Forbidden`.

3. What is the difference between authentication and authorization?

Authentication answers **“who are you?”** Authorization answers **“what may you do?”** You cannot decide the second without first settling the first.

Aspect	Authentication (AuthN)	Authorization (AuthZ)
Question	Who is making the request?	What can this identity do?
Runs	Once per login or token issue	On every protected operation
Depends on	Credentials or a signed token	An identity plus policy
Failure code	`401 Unauthorized`	`403 Forbidden`
Examples	Password login, OTP, JWT verify, mTLS	Role check, scope check, ACL lookup
If compromised	Impostor gets in	Privilege escalation

Memory hook: auth**N**tication is your **Name**, author**Z**ation is the **Zones** you may enter.

4. Which comes first, authentication or authorization, and why?

Authentication, always. Authorization is a decision about an identity, so the system must know who the caller is first. The data flow fixes the order: AuthN takes credentials and outputs an identity, then AuthZ takes that identity and outputs allow or deny. The output of step one is the input to step two.

5. Give a real-world example that explains both.

A hotel shows both steps clearly:

- **Check-in:** you show ID and booking, staff confirm you are the guest. That is authentication.
- **Key card:** it opens your room, the gym, and the pool, but not other rooms or staff areas. That is authorization.

Identity is checked once at reception; the card is checked at every door. In software, the login is check-in, the token is the key card, and each protected endpoint is a locked door.

6. What is Single Sign-On (SSO)?

SSO lets a user log in once with a central identity provider (IdP) and then reach many applications without logging in again. The IdP (Okta, Azure AD, Google Workspace) authenticates the user and issues a token or assertion that each app trusts, so the apps never see the password. Common protocols are **SAML** and **OIDC** (OAuth 2.0 on top).

Teams use it for one set of credentials, central MFA and offboarding, and less password reuse.

Trade-off: the IdP becomes a single point of failure and a high-value target. If it is down, every app is locked out; if it is breached, the blast radius is every app.

7. What are the different authentication methods?

Each method proves identity differently and fits a different situation.

Method	How it works	Best for
Username and password	Compare a secret against a stored hash	Primary human login
API key	Long random secret tied to an app	Server-to-server calls
Session-based	Server stores a session, cookie holds the id	Classic web apps
Token-based	Signed token verified statelessly	APIs, mobile, microservices
Certificate (mTLS)	Both ends present X.509 certs	Service-to-service, high security
Biometric	Fingerprint or face, checked on device	Device unlock, passwordless
OTP	Short one-time code	Second factor
Social login	Sign in via Google, Apple, GitHub	Consumer apps, low signup friction
MFA	Two or more factor types together	Anything sensitive

Username and password. The user sends a secret, the server compares it against a stored hash, never plaintext. Hash with a slow KDF (bcrypt, scrypt, or Argon2) plus a per-user salt. A plain fast hash like SHA-256 is brute-forced at scale, and a generic “invalid credentials” avoids leaking which usernames exist.

API key. A long random secret tied to an application or service account, sent in a header. It identifies the app, not a person, and suits machine-to-machine calls. Keys are long-lived, so scope them narrowly, rotate them, and never ship them in client-side code or URLs.

Session-based. After login the server stores a session record and returns a session id in a cookie. The server is stateful, so it can revoke a session instantly by deleting it. The store must be shared to scale, and cookies bring CSRF risk, so add a same-site policy.

Token-based. After login the server issues a signed token (usually a JWT) that the client sends on every request. The server verifies the signature and stays stateless, which fits APIs and microservices. The catch is that a signed token cannot be revoked before it expires, so keep it short-lived with a refresh token.

Certificate-based (mTLS). The client presents an X.509 certificate and the TLS handshake verifies both ends, so no shared secret travels over the wire. Common for service-to-service traffic in a mesh. The cost is running a PKI: issuing, rotating, and revoking certificates.

Biometric. A fingerprint or face, checked on the device. It usually unlocks a local private key (WebAuthn or passkeys); the biometric itself never leaves the device, which matters because you cannot change a fingerprint once it leaks.

OTP. A short one-time code valid for a minute or two, either TOTP from an authenticator app or sent over SMS or email. Mostly a second factor. SMS OTP is the weakest form because SIM-swap and interception defeat it, so prefer TOTP or a push approval.

Social login. Sign in with an existing provider (Google, Apple, GitHub) over OAuth 2.0 / OIDC, so the user creates no new password. Good for low signup friction, at the cost of depending on the provider and handling account linking.

MFA. Requires two or more different factor types together, so one stolen credential is not enough. Two of the same kind (two passwords) is not MFA; the factors must come from different families.

8. How should passwords be stored securely?

Never store a password as plaintext or as reversible encryption. Store a salted hash from a slow password-hashing function.

- Use a function built for passwords: **Argon2id** (first choice), bcrypt, scrypt, or PBKDF2.
- Add a **per-user random salt** so identical passwords hash differently and rainbow tables are useless.
- Tune the work factor so one verification takes tens of milliseconds. Slow is the point.
- Optionally add a **pepper**, a secret kept outside the database.
- Rehash on next login when you raise the parameters.

> **Trap**
>
> Fast hashes (MD5, SHA-256) and encryption are both wrong here. Fast hashes are brute-forced cheaply, and encryption is reversible the moment the key leaks.

9. What is password hashing?

Hashing runs the password through a **one-way function** to a fixed-length digest that cannot be reversed. To check a login, you hash the input and compare digests, never decrypt.

Password hashing differs from ordinary hashing in one way: it is **deliberately slow and salted** (Argon2, bcrypt), so guessing at scale is expensive. A general-purpose hash like SHA-256 is fast, which is exactly what you do not want for passwords.

10. Difference between hashing and encryption.

The core split: **hashing is one-way, encryption is two-way**.

Aspect	Hashing	Encryption
Direction	One-way, cannot reverse	Two-way, decrypt with a key
Uses a key	No	Yes
Purpose	Verify a password or integrity	Protect data you need back later
Output	Fixed-length digest	Ciphertext, size tracks input
Examples	SHA-256, Argon2, bcrypt	AES, RSA, ChaCha20

> **Interview trap**
>
> “Encrypt the password” is a red flag. Passwords are **hashed, not encrypted**, because you never need the original back.

11. What is a brute-force attack?

A brute-force attack tries many credential guesses until one works. Common variants:

- **Pure brute force:** try every combination.
- **Dictionary attack:** try common passwords from a list.
- **Credential stuffing:** reuse username-password pairs leaked from other breaches.
- **Password spraying:** try one common password against many accounts, to dodge lockouts.

12. How do you protect against brute-force attacks?

Layer several defences, because no single one is enough:

- Slow, salted password hashing so each guess is costly.
- **Rate limiting** per IP and per account.
- Exponential backoff or a temporary lockout after repeated failures.
- CAPTCHA once failures cross a threshold.
- **MFA**, which makes a correct password alone useless.
- Block breached passwords against known-leaked lists, and monitor for login spikes.

> **Trap**
>
> A hard permanent lockout can be turned against you, since an attacker can lock out real users as a denial-of-service. Prefer backoff plus MFA.

13. What is a cookie?

A cookie is a **small key-value string the server sets** with `Set-Cookie`. The browser stores it and sends it back automatically on every request to that domain.

Key attributes control safety and lifetime: `Expires` / `Max-Age`, `Domain` and `Path`, `Secure` (HTTPS only), `HttpOnly` (hidden from JavaScript), and `SameSite` (limits cross-site sending). A cookie usually carries a session id or small preferences.

> **Trap**
>
> A session cookie should be `HttpOnly`, `Secure`, and `SameSite`. `HttpOnly` blocks theft through XSS; `SameSite` blunts CSRF.

14. Difference between cookie and session.

They are not the same kind of thing. A **cookie is client-side storage**; a **session is server-side state**, usually pointed to by a session id that happens to live in a cookie.

Aspect	Cookie	Session
Lives on	The browser (client)	The server (session store)
Holds	Any small value, or a session id	User state for the visit
Visible to user	Yes, in the browser	No, only the id leaves the server
Size	About 4 KB	Limited by server storage

15. Difference between cookie and local storage.

Both live in the browser, but they behave differently.

Aspect	Cookie	localStorage
Sent to server	Yes, automatically each request	No, stays in the browser
Capacity	About 4 KB	About 5 to 10 MB
Lifetime	Until its expiry	Until code or the user clears it
Read by JavaScript	No, if `HttpOnly`	Always
Typical use	Session id, auth	Non-sensitive app data, caching

> **Trap**
>
> Avoid keeping tokens in `localStorage`. It is always readable by JavaScript, so one XSS bug leaks the token. An `HttpOnly` cookie is safer.

16. Difference between cookie and session storage.

Same picture as cookie v/s localStorage, with one change: `sessionStorage` **is cleared when the tab closes** and is scoped to that one tab.

Aspect	Cookie	sessionStorage
Lifetime	Until expiry, survives a restart	Until the tab closes
Scope	Whole browser for the domain	One tab only
Sent to server	Yes	No

17. Session v/s token.

The classic trade-off is **stateful sessions v/s stateless tokens**.

Aspect	Session (stateful)	Token / JWT (stateless)
Server stores state	Yes, a session store	No, the token carries claims
Verify a request	Look up the session id	Check the signature
Revoke early	Easy, delete the session	Hard, valid until expiry
Scaling	Needs a shared store	Nothing shared to check
Best for	Server-rendered web apps	APIs, mobile, microservices

Rule of thumb: want instant revocation and server control, pick sessions. Want stateless scale across services, pick tokens with short expiry plus a refresh token.

18. Access token.

An access token proves the client is **allowed to call an API on the user's behalf**. It carries scopes and is short-lived, usually minutes. It is sent on each API call, normally as a bearer token, and kept short-lived so a leaked one expires quickly.

19. Refresh token.

A refresh token is a **long-lived credential used to get a fresh access token** when the old one expires, without asking the user to log in again. It is sent only to the auth server, never to normal APIs, and should be stored carefully.

> **Trap**
>
> A refresh token is high value. Use **rotation** (a new one on each refresh) with **reuse detection**, so a stolen one is caught.

20. ID token.

An ID token is an **OIDC JWT that tells the client app who the user is**: name, email, subject id, and when they logged in. It answers authentication (identity), in contrast to the access token, which is about authorization (API access).

> **Trap**
>
> Do not send an ID token to an API to gain access. It is meant for the client, not as an API credential. Use the access token for that.

21. Bearer token.

A bearer token is any token where **whoever holds it can use it**, with no extra proof of possession. It is sent as `Authorization: Bearer <token>`. Access tokens are usually bearer tokens. It is convenient but risky: like cash, whoever steals it can spend it. That is why you always use TLS and short expiry, and why higher-security setups bind the token to the client with mTLS or DPoP (sender-constrained tokens).

22. What is JWT, and why do we need it?

A JWT (JSON Web Token) is a **compact, URL-safe, signed token that carries claims as JSON**. Any service with the key can verify it, so no server lookup is needed.

It makes authentication **stateless**: there is no shared session store, each service verifies the signature locally and reads the claims (user id, scopes, expiry), and it scales cleanly across APIs, mobile apps, and microservices where a central session store would be a bottleneck.

> **Interview trap**
>
> **Signed is not encrypted.** The payload is only base64url-encoded, so anyone can read it. Never put secrets in it, and remember it cannot be revoked before it expires.

23. What is the structure of a JWT?

Three parts, dot-separated, each base64url-encoded: `header.payload.signature`.

```
header    -> {"alg":"HS256","typ":"JWT"}
payload   -> {"sub":"user-42","exp":1716003600,"scope":"read:profile"}
signature -> HMACSHA256( base64url(header) + "." + base64url(payload), secret )
```

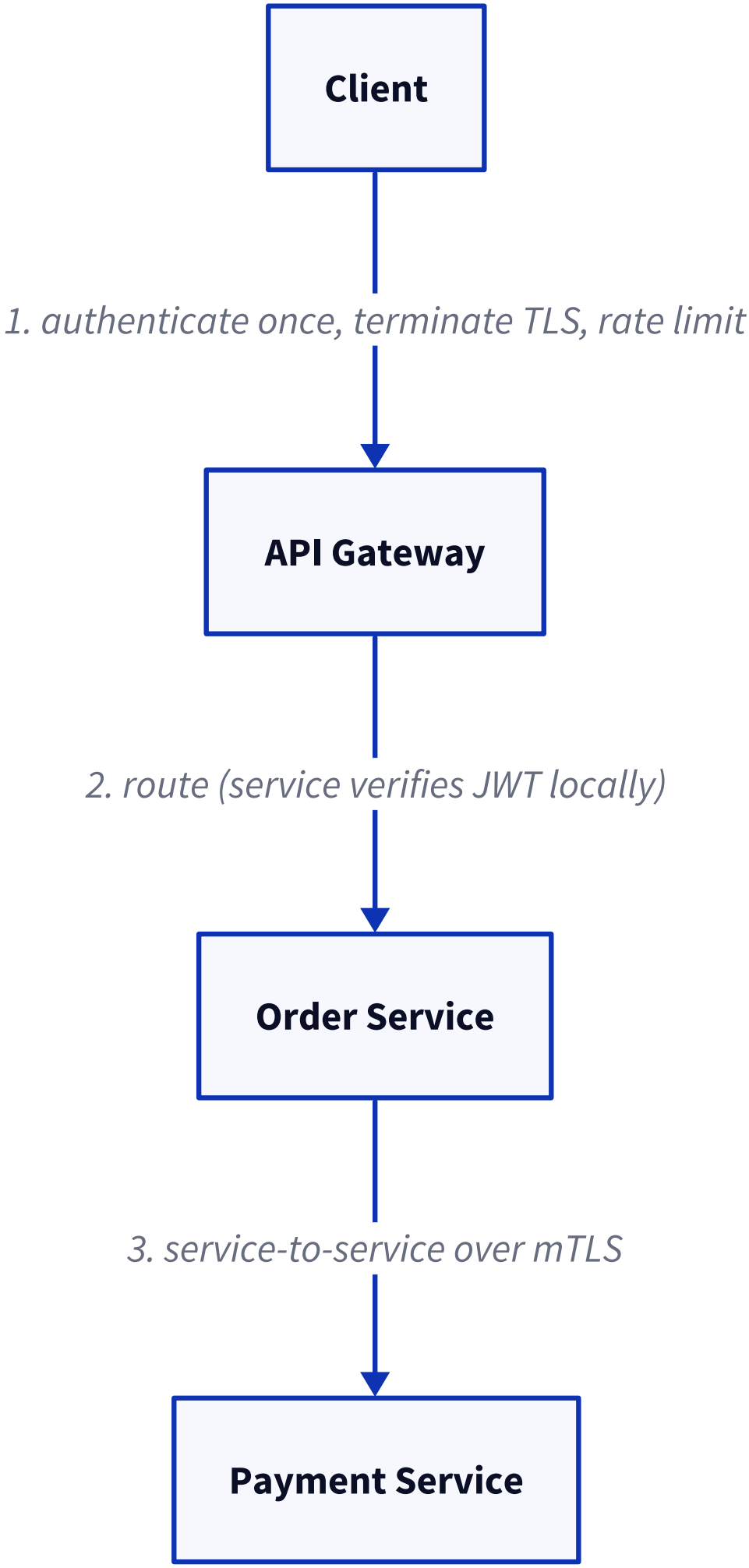
- **Header:** the signing algorithm (`alg`) and type.

- **Payload:** the claims, such as `sub`, `iss`, `aud`, `exp`, `iat`, and scopes.
- **Signature:** the header and payload signed with a secret (HMAC) or a private key (RSA / ECDSA).

> **Interview trap**
>
> Always **pin the expected algorithm**. The `alg: "none"` and algorithm-confusion attacks (tricking the server into treating an RSA public key as an HMAC secret) are real.

24. How is authentication handled in microservices?

Authenticate once at the edge, then let each service verify a token locally. The gateway or an auth service checks the credentials and issues a JWT; every downstream service then verifies the signature and claims on its own, without calling back to the auth service.


Client
1. authenticate once, terminate TLS, rate limit
API Gateway
2. route (service verifies JWT locally)
Order Service
3. service-to-service over mTLS
Payment Service

Service-to-service calls use mTLS or short-lived service tokens for their own identity, while user identity is carried forward inside the token.

> **Trap**
>
> Do not trust identity headers coming from outside the mesh. An internal service should **verify the token itself**, not blindly believe an `X-User-Id` header, in case a request bypasses the gateway.

25. API Gateway authentication.

The API gateway is the **single entry point that authenticates every incoming request** before routing it, so individual services do not each reimplement auth. At the edge it validates the token or API key, terminates TLS, applies rate limits and quotas, and forwards a trusted identity (verified claims) to internal services.

> **Trap**
>
> Centralizing auth at the gateway is convenient, but under zero-trust the internal services should still verify the token. Network position alone is not proof of identity.

26. How do APIs authenticate requests?

The client sends a credential with the request and the server verifies it. Common mechanisms:

- **Bearer token** (JWT or opaque) in the `Authorization` header.
- **API key** in a header, for machine-to-machine calls.
- **mTLS client certificate** at the transport layer.
- **HMAC-signed request** (as in AWS SigV4): the client signs the request with a shared secret, which stops tampering and replay.
- **OAuth 2.0 access token** carrying scopes.

Whichever you use, verify the token's signature, expiry, audience, and scope, and only over TLS.

27. Session fixation.

The attacker gets a session id set **before login** (for example, by planting one), the victim then logs in on that same id, and the attacker rides the now-authenticated session.

Defence: regenerate the session id on login, and on any privilege change, so the pre-login id becomes useless.

28. Session hijacking.

The attacker **steals a valid session id or token** (through XSS, network sniffing, or a leak) and reuses it to act as the user.

Defence: TLS everywhere, `HttpOnly` and `Secure` cookies, short session lifetimes, and rotating the session id periodically.

29. Cookie theft.

Stealing the session or auth cookie, usually through **XSS or an insecure channel**, then replaying it.

Defence: `HttpOnly` (blocks JavaScript access), `Secure` (HTTPS only), `SameSite`, and short expiry.

30. How would you authenticate public APIs?

For an externally exposed API, identify the caller with an **API key** and, when acting for a user, an **OAuth 2.0 access token with scopes**. Add rate limits and quotas per key, and serve only over TLS. For higher assurance, use HMAC-signed requests or mTLS. Support key rotation and revocation from day one, and treat any truly unauthenticated endpoint as still needing rate limiting against abuse.

31. How would you authenticate third-party integrations?

Use **OAuth 2.0** so the third party receives a scoped, revocable access token instead of your users' passwords. For server-to-server integrations, use the **client credentials grant**. For inbound webhooks, verify an **HMAC signature** on the payload to prove it came from the partner. Scope permissions to the minimum, rotate secrets, and keep every grant revocable.

32. How would you authenticate gRPC requests?

gRPC runs on HTTP/2, so it authenticates on two layers:

- **Channel level:** mTLS, where both sides present certificates. Common for service-to-service identity.
- **Call level:** a token in the request metadata, for example `authorization: Bearer <jwt>`, checked by a server interceptor that runs before the handler.

In practice you combine them: mTLS proves the calling service, and the JWT carries the user identity.

33. A user's JWT is stolen. What happens?

Because a JWT is a bearer token, **whoever holds it can act as the user until it expires**. Stateless verification means the server cannot tell a stolen token from a real one: the signature still checks out, so it is accepted. The damage is bounded by the token's **scope and lifetime**, which is the whole reason access tokens are kept short-lived.

Mitigation: short TTL, sender-constrained tokens (mTLS or DPoP) so a stolen token is useless without the client key, and refresh-token rotation with reuse detection.

34. How would you revoke a JWT immediately?

A plain JWT cannot be revoked, because it is valid until `exp`. To force revocation you add a little state back:

- **Short access tokens plus a revocable refresh token:** kill the refresh token and access dies within minutes.

- **Token denylist:** store revoked token ids (`jti`) in a fast store like Redis and check each request against it.
- **Per-user token version:** keep a "tokens valid after" timestamp and bump it to invalidate every token issued earlier.

Trade-off: all three reintroduce a lookup on each request, so you give up some of the statelessness that made JWT attractive.

35. How would you log out from all devices?

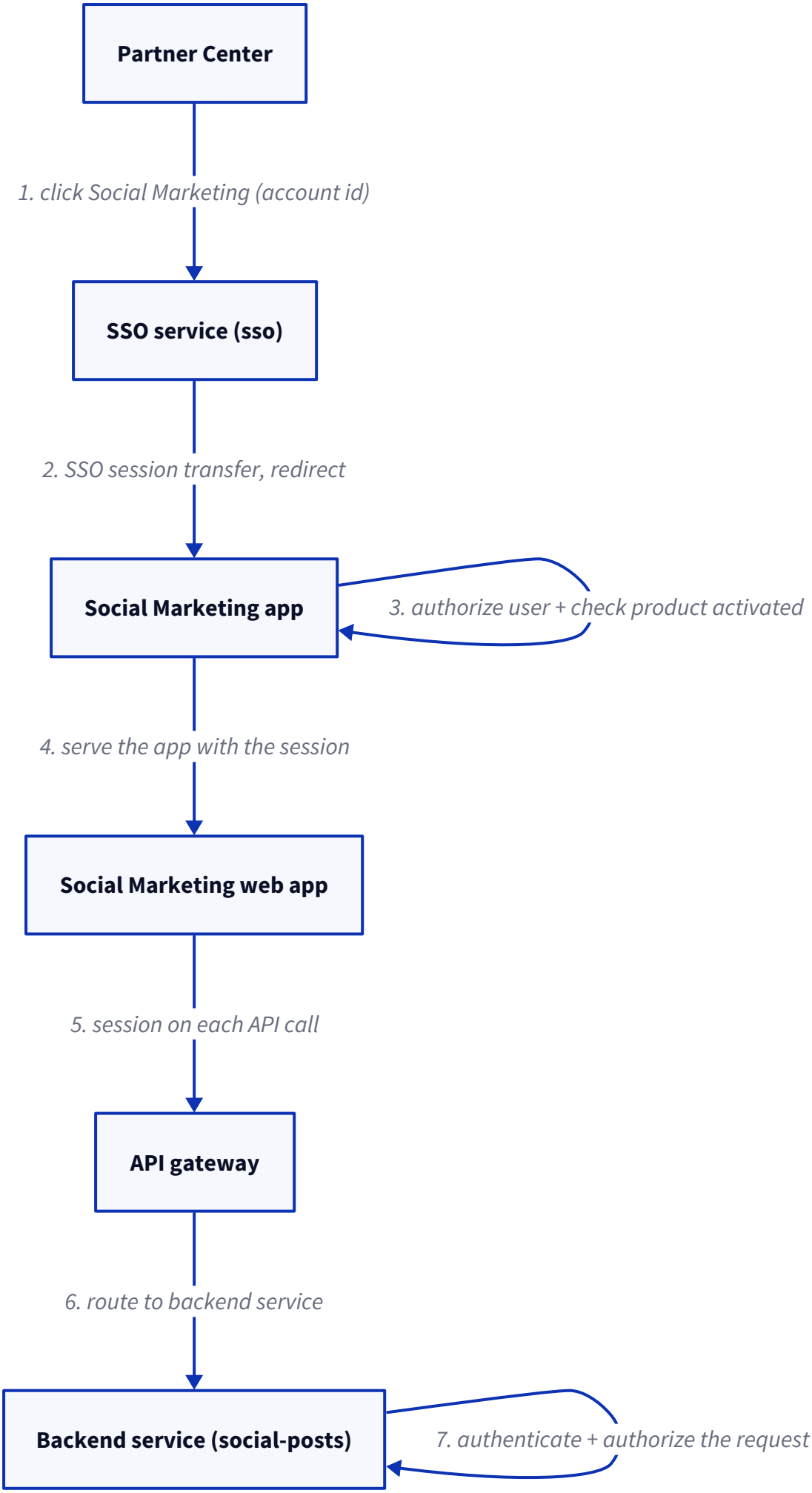
Invalidate every token or session for that user at once. The common ways:

- **Bump a per-user token version** (or minimum issued-at) in the database, then reject any token issued before it. Every device must log in again.
- **Delete all the user's sessions** from the session store, for stateful sessions.
- **Revoke all refresh tokens** for the user, so no device can mint a new access token.

Vendasta-specific questions

1. A user logged into Partner Center opens Accounts, Manage Accounts, selects a business, and clicks the Social Marketing button. Walk through the complete request flow, with authentication and authorization at every hop.

The key thing to get is that clicking the button does not open the Social Marketing app directly. What happens is Partner Center hands the browser over to our **SSO service**, which uses single sign-on to carry the user's existing login across into the Social Marketing app. The app then checks that this user is actually allowed to open this business, and that the business even has the product, before it shows anything. From there the UI's API calls go through the **API gateway** to the backend, and each backend service authenticates and authorizes the request on its own.


Partner Center
1. click Social Marketing (account id)
SSO service (sso)
2. SSO session transfer, redirect
Social Marketing app
3. authorize user + check product activated
4. serve the app with the session
Social Marketing web app
5. session on each API call
API gateway
6. route to backend service
Backend service (social-posts)
7. authenticate + authorize the request

Let me walk it hop by hop:

1. **Already logged in.** The user signed in to Partner Center earlier, so the browser is already holding a valid session that IAM, our identity service, issued.
2. **Click the button.** Partner Center does not link straight to the SM app. It sends the browser to the SSO service's service-gateway endpoint, passing along the business they picked (the account group id).
3. **Single sign-on handoff (authN).** The SSO service sees the existing login and does a session transfer, so it carries the authenticated identity across to the Social Marketing app and drops the browser there already logged in. The identity crosses through the SSO session, not by sticking a token in the URL.
4. **Open checks in the SM app (authZ).** Before it shows anything, the app runs two checks: an IAM permission check that this user may touch this business, and a check that the business actually has Social Marketing activated. If either fails, it refuses.
5. **Serve the UI.** The app serves its web front-end to the browser, carrying the session.
6. **API calls.** The front-end calls the backend through the API gateway, sending the session as a bearer token on each call.
7. **Backend enforces (authN + authZ).** Each backend service authenticates the session and authorizes the specific action against the business. The gateway is not the security boundary; every service checks for itself.

Which services are involved.

Service	Role in this flow
Partner Center	Where the user clicks to launch Social Marketing
SSO service	Single sign-on and the session transfer between apps
IAM	Issues and validates the user's identity, and answers the permission check
Social Marketing app	Checks access and product activation, then serves the UI
API gateway	Fronts the UI's backend API calls
Backend services (for example social-posts)	Authenticate and authorize each API request

Every credential exchanged.

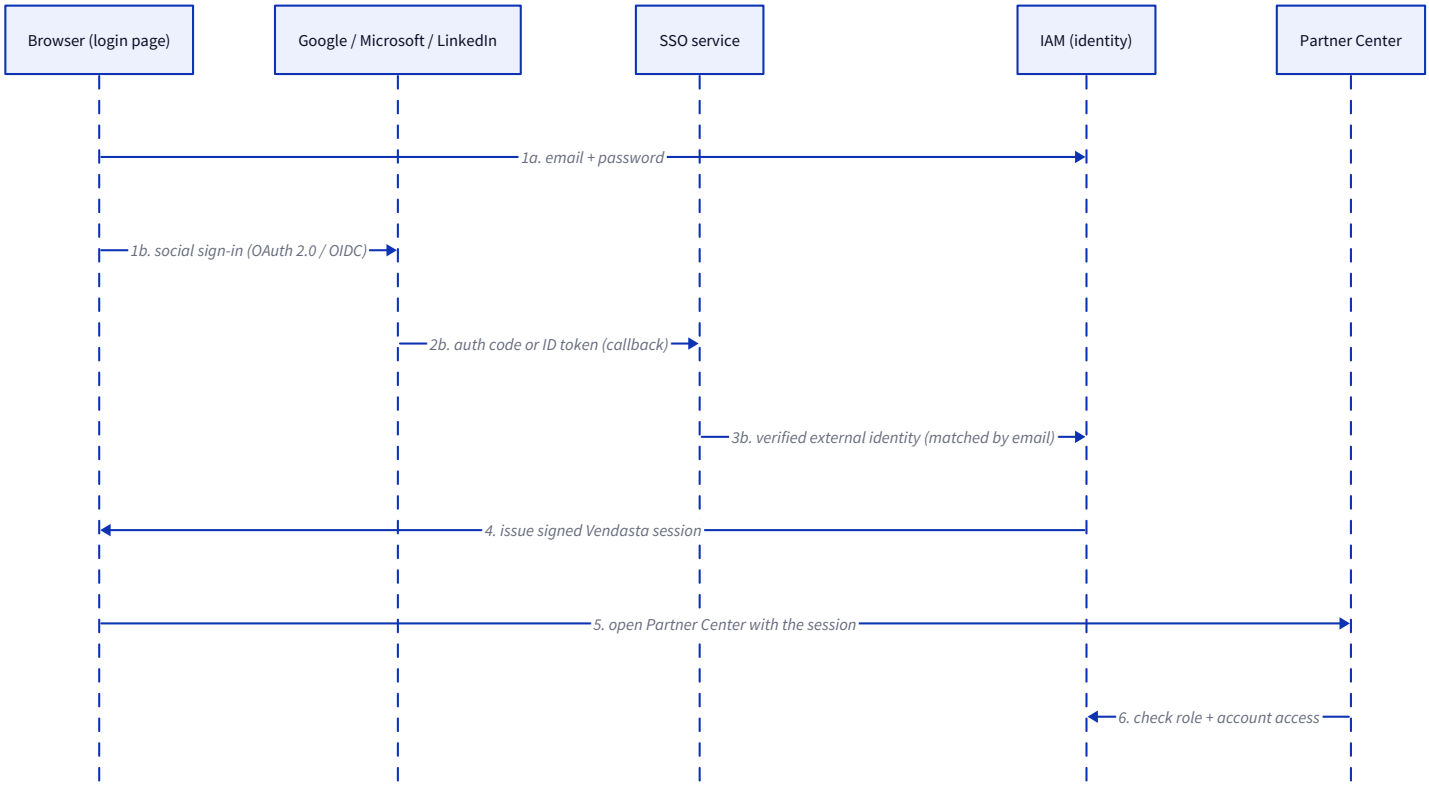
Credential	Purpose
IAM user session	Proves who the user is
SSO session	Carries the login across from Partner Center to the SM app
Bearer session on API calls	Authenticates each backend request
Service token	Authenticates service-to-service calls (see scenario 6)

What decides that this user may open Social Marketing for this business.

It comes down to the **IAM permission check on the business (account group)**. It uses the user's **role** (partner staff, agency user, or business user) and which partner and businesses they belong to. **Read** access opens the app; **write** access is what later authorizes publishing. And separately, the business has to have the product activated, or the app refuses regardless of role.

2. A new user opens Partner Center and is sent to the login page. It offers email and password, Google, Microsoft, and LinkedIn. How is the user authenticated and authorized for each?

All four paths actually end up in the same place. **IAM**, our identity service, is the one authority that verifies who you are and issues the Vendasta session. The only thing that differs is how you prove yourself up front. Email and password, IAM checks directly. Google, Microsoft, and LinkedIn are **federated**: you prove who you are to that provider over OAuth or OIDC, our **SSO service** brokers the exchange, and then IAM maps that external identity to a Vendasta user and issues the session. After that point, every path is identical.



Login method	Protocol	Who verifies the user
Email and password	Vendasta password login	IAM
Google	OpenID Connect	Google, result read by IAM
Microsoft	OpenID Connect	Microsoft, result read by IAM
LinkedIn	OAuth 2.0 (OIDC available)	LinkedIn, result read by IAM

The flow for each option.

Email and password. The user hands their credentials to IAM, IAM checks them against the stored password hash, and on success it issues a signed Vendasta session. No third party involved.

Google and Microsoft (OpenID Connect). The user gets redirected to the provider and signs in there, and the provider hands back a signed ID token that proves who they are, so we never see the password.

LinkedIn (OAuth 2.0). Same shape, we get an authorization code, exchange it, and read the verified profile, mainly the email. LinkedIn also offers an OIDC mode now.

For all three social options, our SSO service runs the OAuth or OIDC exchange and hands the verified external identity over to IAM.

How OAuth and OpenID Connect fit.

The way I frame it: OAuth 2.0 is what lets the user authorize us to learn who they are at the provider, without sharing a password. OpenID Connect adds the identity layer on top, a signed ID token that carries the verified email and a stable subject id. Google and Microsoft use OIDC, LinkedIn is OAuth 2.0 with an OIDC option. In every case the provider does the authentication and we just consume the verified result.

How identity, roles, and access are decided.

Identity is not decided by how you logged in. Once the provider, or the password check, proves who you are, IAM issues one signed Vendasta session that represents you, the same session we use across all the apps in scenario 1. For social login, IAM matches the provider's verified email to an existing Vendasta user, that is the account linking. Roles and access do not come from the login method either. Once IAM knows who you are, your role (partner staff, agency user, business user, or a service account) and which partner and businesses you belong to come from IAM and the partner service, and those associations decide what you can see and do.

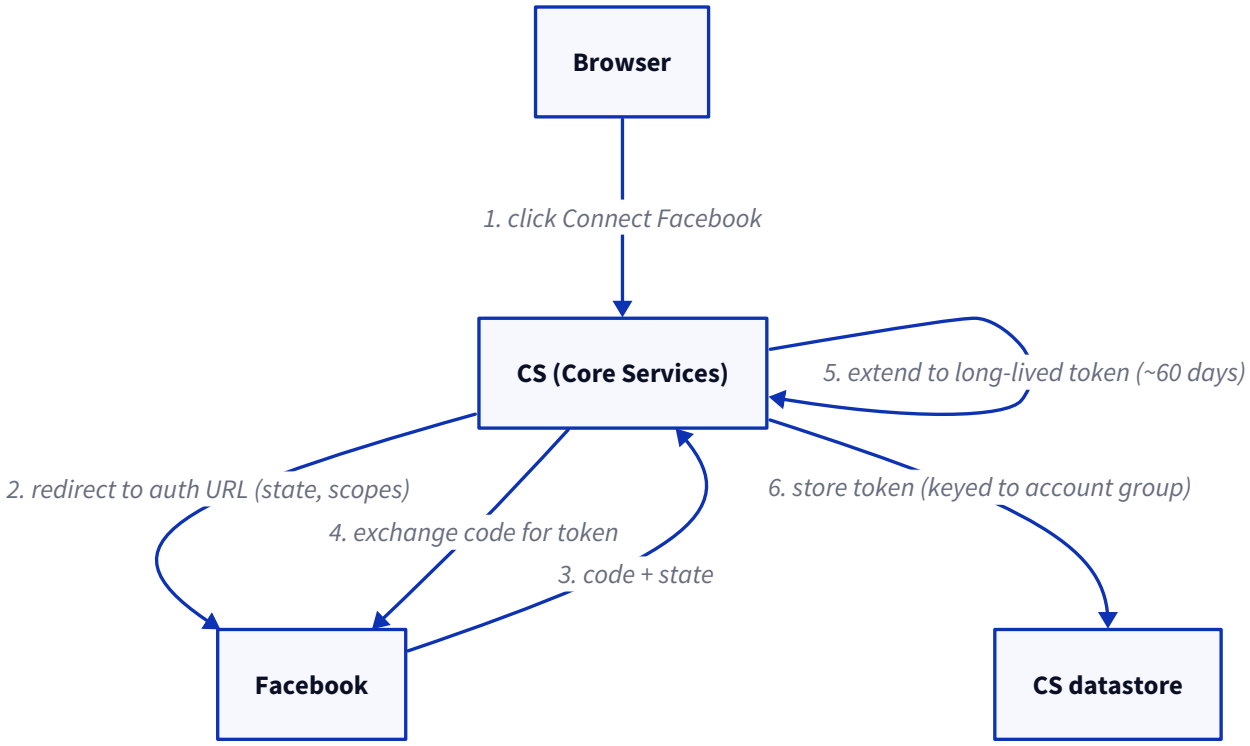
How the user reaches Partner Center and other products.

The browser lands on Partner Center carrying the session, Partner Center trusts IAM's session and shows whatever the user's role and associations allow. And opening any product from there, like Social Marketing, just runs the same per-business IAM permission check and product-activation check I described in scenario 1.

3. Walk through the full OAuth flow when a business connects a Facebook Page or Instagram account, from the click to the stored token.

The first thing I would say is that Facebook and Instagram look like one feature but they actually run on **two separate OAuth stacks with two separate token stores**. Facebook goes through our older **Core Services, CS**; Instagram has its own Go microservice.

So for Facebook, through CS:



Instagram follows the same authorization-code idea, but the standalone instagram service runs its own OAuth2 flow and stores the token in its own VStore, keyed by partner, business, and user. It never touches the CS store. Either way the user never sees a token, and they only need to reconnect if the token later expires.

Which grant type, and why that one for this case?

The **authorization code grant**. It is the right fit for a server-side web app connecting on a user's behalf, because the code gets exchanged for the token **on the server**, so the token never passes through the browser, and the network can tie the exchange to our app's `client_secret` and the registered `redirect_uri`. X is the odd one out, it is still on **OAuth1** three-legged, and those tokens do not expire.

What is the state parameter doing, and what breaks without it?

`state` ties the callback back to the request that started it, which is what stops **login CSRF**, where an attacker gets a logged-in user to finish an OAuth callback the attacker started and ends up connecting the attacker's account. Now, to be precise about us: in the **Instagram service** `state` is a real CSRF nonce, a random uuid we store server-side and look up on the callback. But in the older **CS Facebook flow**, `state` is really just a **data carrier**, a base64 JSON blob with the next URL and a profile id, and it is not checked server-side, so that flow is leaning on the exact redirect-URI match instead. Without a real `state` check, that login-CSRF door is open.

Why is the token held server-side in Core Services rather than in the browser?

So the user never holds a network token, and so an XSS or a stolen browser cannot walk off with the power to post as the business. CS also **owns the working network integrations**, so keeping the tokens next to the code that uses them was the safe call during the migration. The user only ever triggers a reconnect.

What does redirect URI validation protect you from?

It stops an attacker **swapping in their own redirect** and catching the authorization code at an address they control. For us it is an **exact-match** check: Facebook rejects any callback URI that does not match the one registered in the app dashboard, and the Instagram service uses a fixed per-environment redirect rather than building one from the incoming request. That exact match is what makes a stolen code useless anywhere else.

4. A post scheduled two weeks out fails at publish time because the token expired. Walk me through how you detect that, and why the design turns the post back into a draft instead of retrying.

So the schedule runs as a **durable Temporal workflow, one per post per network**. It sleeps until the due time, wakes up, re-reads the post so it picks up any edit, then calls the publish activity. An expired token shows up **reactively**, when the network API rejects that call with an auth error. Retrying does not help, because the token genuinely needs a **human to reconnect the account**, so we turn the post back into a **draft** with its text and image intact and prompt the user to reconnect and resubmit. Burning retries on a dead token gets you nothing, and in the timeout case a blind retry actually risks a duplicate public post.

Access token v/s refresh token: why not silently refresh and publish anyway?

For most of our networks there is nothing to silently refresh. Facebook uses a long-lived token, around 60 days, and once it expires the user has to reconnect, there is no refresh-token exchange. Only **LinkedIn** actually holds a refresh token and refreshes on its own. So “just refresh and publish” is not on the table for Facebook, Instagram, or X, and the honest recovery is a reconnect, which is exactly why the draft fallback exists.

How do you tell an expired token apart from a transient network error, so you don’t fall back when you should retry?

By **classifying the error**. A transient failure, a timeout or a 5xx or a rate limit, is retryable, so the workflow retries within bounded limits. An auth failure, an expired or revoked token, is terminal for this attempt, so it goes straight to a draft instead of wasting retries.

> **The hard case**
>
> A network that **accepts the post and then times out** must not be retried, or you publish a duplicate. Those errors are classified as not-worth-retrying. This error classification is the core of the reliability work.

Do you refresh proactively before expiry, or react on failure? Trade-off?

We are **mostly reactive**, we find out a token is bad only when the publish call fails. **LinkedIn is the exception** and refreshes proactively. Proactive saves you a failed publish and a reconnect prompt, but it costs you extra token-introspection and refresh machinery per network. Reactive is simpler and cheaper, but the first the user hears of it is a failed post. The pragmatic split is to refresh proactively only where the network actually gives you a refresh token, and reconnect everywhere else.

Instagram and X hold their own tokens. Does their expiry path differ from the CS-held ones?

Yes. Facebook, LinkedIn, X, GBP, YouTube, and TikTok tokens all live in **Core Services**, and social-posts hands their publish to CS. **Instagram runs its own microservice** with its own token store and publish workflow, so its expiry gets found inside that service, which also fires a **broken-token event** so the UI can prompt a reconnect. **X is different again**, on OAuth1 its tokens do not expire, though they can still be revoked.

5. The same backend serves every partner and every business. How do you guarantee a marketer at agency A can never publish for, or read analytics of, a business under agency B?

So the tenant key is the **account group id (AGID)**, which is the id of one business, and it rides on every request. The isolation does not come from trusting that id, though. It comes from **checking it against the caller's authority in central IAM** before we touch any data. Each handler calls its auth check, `AccessAccountGroup`, with the AGID and the action; that resolves the business's partner and calls **IAM v2** `AccessResource`, which compares the caller's role and partner claims, read from their session and not from the request, against the resource. Agency A's marketer only carries claims for A's businesses, so a request for a B business just gets denied.

Where does the tenant-scope check live: gateway, service layer, or data layer?

The **service layer**. The gRPC interceptors work out **who** the caller is, the identity, but they do not decide resource access. Each handler makes the authorization call itself, right before it reads or writes. The data layer just filters by whatever AGID it is handed, it does not re-check authority.

How do you stop an IDOR where a caller just passes another account group's id?

The AGID in the request is **never trusted on its own**. IAM pulls the caller's own claims from their session and checks them against that AGID, so passing a stranger's id just fails the `AccessResource` check.

> **Honest nuance**
>
> This check has to be added in **every handler by hand**, and honestly a few RPCs have historically missed it, which is a real IDOR risk. The proper fix is to make the check impossible to forget (a default-deny wrapper), not to rely on discipline.

The account group id travels with every post. How is that scope enforced on a read query, not just trusted?

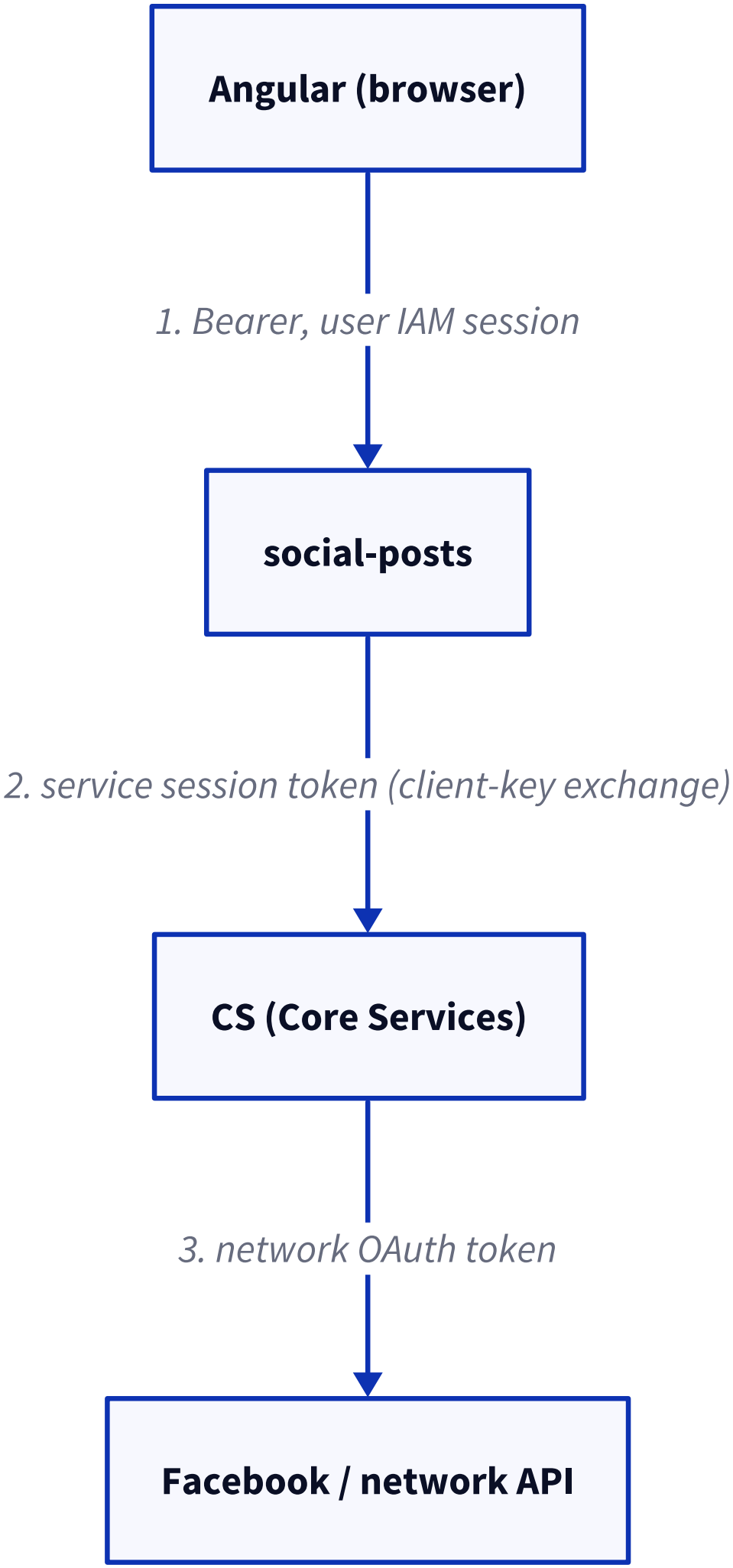
The read query does filter by AGID, for example an Elasticsearch term query on `account_group_id`, but **that filter is scoping, not the security boundary**. The boundary is the `AccessResource` check that ran first and proved the caller owns that AGID. If a handler skipped the check, the query would happily hand back another tenant's data, so it is the authorization call, not the query filter, that actually enforces isolation.

authN v/s authZ in this exact request, concretely.

authN is the IAM interceptor validating the caller's session JWT signature and setting the identity on the context. **authZ** is `AccessAccountGroup`, then `AccessResource`, checking that this identity may take this action on this AGID. First proves who, second proves whether they may.

6. Trace identity across the hops: Angular to social-posts to CS to the network API. How does each hop authenticate, and how is the caller's authority propagated?

Every internal hop is authenticated with an **IAM-issued session JWT** carried as an `Authorization: Bearer` header. It is app-layer, not transport-layer.


Angular (browser)
1. Bearer, user IAM session
social-posts
2. service session token (client-key exchange)
CS (Core Services)
3. network OAuth token
Facebook / network API

One detail: social-posts is gRPC-native, so a bridge called **bifrost** takes the Angular app's HTTP/JSON call and turns it into a gRPC call, carrying the header straight through.

What proves social-posts is allowed to call CS at all?

Its **private key**. social-posts signs a short-lived proof with a long-lived **client key** that was provisioned out of band, a Kubernetes secret, IAM verifies that signature against the public key it holds for that service account and mints a session token, and then CS's IAM interceptor verifies that session token. Nothing about network position or cloud identity is checked; the key is the whole proof.

mTLS, service tokens, or GCP service accounts here? What does Vendasta actually use?

Minted IAM service tokens. Not mTLS, and not GCP service-account identity tokens. gRPC even runs **without TLS** between our services, so the trust sits entirely in the app-layer signed JWT. The one exception is an inbound external partner that shows up with a Google service-account token, which social-posts re-mints into a Vendasta IAM token before it trusts it.

Do you pass the end-user context through, or does each service re-derive authority from scratch?

Both identities travel. The direct caller, the microservice, is in the `Authorization` header, and the original caller, the human user, rides in a separate `x-caller-identifier-session` header. Downstream authorization runs against the **effective caller**, which is the original user when it is present, so a service acts with the user's authority, not its own broad service rights.

Where do gRPC interceptors fit in enforcing this?

They are the **choke point**. A fixed unary interceptor chain runs on every RPC: timeout, logging, error mapping, then the **IAM interceptor** that authenticates the bearer token and populates the caller identity, then a **caller-id interceptor** that sets the effective caller on the context. Authentication happens in the interceptor, and the per-resource authorization, `AccessResource`, happens in each handler using the identity the interceptor set up.

7. The product calls Vertex AI and OpenAI. How does each call authenticate, and why does the write-up prefer Vertex's GCP identity over a separate provider credential?

They authenticate in two different ways, and that difference is really the whole point. **Vertex AI (Gemini)** uses the workload's **GCP identity** through Application Default Credentials, so there is no static key in the code, the running service account is the credential. **OpenAI** uses a **static API key** that we read from GCP Secret Manager at startup. The reason we prefer Vertex is exactly that difference: a service-account identity means no long-lived secret to store, leak, or rotate, and it keeps everything under one identity, one billing and quota setup, and inside the GCP trust boundary, which matters a lot for a white-label product holding many businesses' data.

GCP service account or workload identity v/s a raw API key.

Vertex takes the service-account path, the platform hands the workload its identity, the tokens are short-lived and GCP rotates them, and nothing static sits in the repo. A raw API key, like OpenAI, is a bearer secret you have to store, read into memory, and rotate by hand. The service-account model is the safer of the two, simply because there is no standing secret to steal.

Where does the OpenAI key live, and how do you rotate it?

In **GCP Secret Manager**, under an environment-specific name, read once at process startup at version `latest`. Rotation today is manual and pretty coarse: you add a new secret version and restart the service, which picks up the new one. There is no live re-fetch and no scheduled rotation in the code, which is a gap worth naming out loud.

Blast radius of a leaked model-provider key.

The OpenAI key is **one shared secret for the whole service**, it is not per business or per partner. So if it leaked, anyone could run up cost and quota on the account and call the model as us, across every tenant, until we rotate it. Vertex has no equivalent static key, so that blast radius just does not exist on the Gemini path.

8. Today CS is the single holder of connections and tokens, and the plan is to move token ownership into each network's own service. What does that change for security and blast radius, and what are the risks during the migration?

Right now CS is a **single high-value store**, it holds the tokens for Facebook, LinkedIn, X, GBP, YouTube, and TikTok for every business under every partner, so a breach of CS exposes all of them at once. Moving each network's tokens into its own service, the way Instagram already is, **shrinks the blast radius per store**: compromise the LinkedIn service and you only expose LinkedIn tokens, not the whole set. The cost is that there are now more stores to secure, so the boundary must not get weaker as we move.

Centralized secret store v/s distributed ownership: which is safer, and safer against what?

Centralized is safer against **drift and inconsistency**, because there is one place to secure, patch, and audit. Distributed is safer against a **single catastrophic breach**, because no one store holds everything. For this product the decision is driven by blast radius, so per-service ownership wins, as long as each service actually applies the same controls instead of reinventing weaker ones.

During coexistence, how do you avoid a window where a token lives in two places and they drift?

You keep **one writer as the source of truth** at any moment and treat the other copy as read-only, syncing one way through events rather than dual-writing. You cut a network over only once its new store is authoritative, and you never trust the deprecated copy once the flag flips. The migration bridge is eventually consistent, so the consumers have to tolerate that and reconcile, not assume both copies agree.

How do you cut over one network at a time without weakening the boundary?

You move networks **behind a flag, one at a time**, with the new service owning both the token and the publish for that network before the old path is retired. Each new store carries the same IAM gating and the same at-rest protection CS had, so the boundary holds as ownership moves, and you can roll a single network back without touching the rest.

9. Beyond the happy path, which parts of the OAuth exchange would you review most carefully for security?

The four that actually bite are the `state` **check**, **redirect URI allow-listing**, whether **PKCE** applies, and **where the token travels**. The theme across all of them is that the redirect is the weak point, because it goes through the user's browser where an attacker's site can sit in the middle.

`state` and CSRF on the callback.

`state` has to be a **server-generated nonce** that you store and then check on the callback, so an attacker cannot forge a completed connect. The honest note for us: the Instagram service does this right (a stored uuid), but the older CS Facebook flow uses `state` as a data carrier and does not verify it server-side, it leans on the redirect-URI match instead. That is the first thing I would tighten.

Redirect URI allow-listing and open-redirect risk.

The callback URI has to be an **exact match against a fixed allow-list**, never built from request input, otherwise an attacker points the redirect at a URL they control and catches the code. Deriving the callback from `request.host`, which is what the CS flow does, is the risky pattern; a hardcoded per-environment redirect, like the Instagram service uses, is the safe one.

PKCE: does it apply here, and why or why not?

PKCE really matters for **public clients**, mobile apps or single-page apps that cannot keep a secret. These flows are server-side **confidential clients** holding a `client_secret`, so PKCE is not strictly required, and the code does not use it. Under OAuth 2.1 it is recommended even for confidential clients, so adding it would be a cheap extra guard against code interception, but it is not the gap that the missing `state` check is.

Token in the URL v/s the body, and how you guard the callback endpoint.

The authorization **code** has to arrive in the redirect query string, there is no avoiding that, but the **token exchange itself must be a POST body over TLS**, never the token in a URL, because URLs leak into logs, browser history, and referrers. You guard the callback by enforcing TLS, checking `state`, matching the redirect exactly, and treating the code as single-use and short-lived.

10. An agency deactivates a marketer who has fifty client businesses open. How quickly does their access actually get cut, and what governs that?

Honestly, **not instantly**. Access is carried by a **stateless IAM session JWT** that our services validate on **signature and expiry only**, and there is **no server-side revocation, no token-version, no denylist** in the service path. So a deactivated marketer keeps working access **until their current token expires**; the cutoff is bounded by the session lifetime, not the moment of deactivation.

> **Honest gap**
>
> There is no mid-life revocation in the services today. To cut access at once you would need the token-version or denylist approach from the Generic section, which is not implemented for user sessions here. Short session lifetimes are what currently bound the exposure.

Stateless JWT (can't revoke mid-life) v/s server-side session: which trade-off did you take?

We took the **stateless JWT** path: every service verifies the signature locally with no lookup, which scales cleanly across a lot of services. The price is exactly this, no instant revocation. A server-side session would let you kill access immediately, but at the cost of a lookup on every request. The pragmatic middle ground, if you need immediate cutoff, is to reintroduce a little bit of state, a per-user "valid after" timestamp.

What happens to a post they scheduled that fires after they are gone?

It still fires. The scheduled post runs later as a Temporal workflow under the **service's own identity**, not the marketer's session (see scenario 16), and authorization was checked back when the post was scheduled, not at publish time. So a post scheduled before deactivation just publishes normally afterwards, unless something explicitly cancels it.

11. A multi-location post fans out to twenty branches. How do you authorize it when the acting user can post to some branches but not all?

Today the check is **once, at the parent brand level, and it is all-or-nothing**. The multilocation handler authorizes the caller against the brand (the parent group) and, if that passes, the fan-out loop creates a post per branch with **no further per-branch check**. So the model is basically assuming that brand-level access implies access to every branch under it.

All-or-nothing, or partial fan-out with the denied ones reported back?

All-or-nothing. Where the code does batch-check account groups, in the drafts fan-out, **any single denial fails the whole request**, and it does not tell you which branch was denied. The IAM SDK actually hands back the successful and failed ids, but the handler throws that detail away, so partial fan-out with a denied-branches report is **not** what happens today.

> **What I'd change**
>
> I would authorize **per branch** using the batch check's success and failure list, publish to the allowed branches, and report the denied ones back to the caller. The building block (a batch check that returns per-id results) already exists; the handler just needs to use it.

Is authorization checked once for the parent post, or per branch?

Once, for the parent brand. There is no per-account-group `AccessResource` call inside the fan-out loop, so branch-level authority is effectively inherited from the brand.

How do you keep one unauthorized branch from failing the whole fan-out?

You cannot cleanly today, because the batch check is all-or-nothing. The fix is the per-branch model I mentioned: treat each branch as its own authorization and let the allowed ones go through independently, the same way each network's publish is already independent.

12. A partner reports a connected account was compromised, or you suspect a token leaked. What is your response, and how do you limit damage in a shared backend?

The immediate move is to **invalidate that one network token and force a reconnect**: mark it broken or delete it in the owning store, CS for most networks, the Instagram service for Instagram, and revoke it at the provider so a copy cannot be reused. Because we store tokens **per business**, killing one does not touch any other business or partner. Then I would rotate anything shared that might have been exposed, and go check what that token actually did.

Revocation and rotation path: how fast can you invalidate one token?

Fast, because it is a **single record**: delete or mark-broken the token in the owning store, and the next publish just falls back to a reconnect prompt. The stronger step is to **revoke at the network itself**, Facebook or LinkedIn, so the string is dead even off our platform. There is no shared token to coordinate, it is one row.

Can one partner's incident touch another partner in a backend that serves everyone?

For the **tokens and the data, no**, they are scoped per business and gated by IAM, so one partner's compromised account does not expose another's. The real shared surfaces are the **shared secrets and the shared hubs**, the one OpenAI key, and CS as a common store. An incident that reached one of those, rather than a single network token, is the one that could cross partners, and that is really the argument for per-service token ownership.

What audit trail tells you what that token did while exposed?

Thinner than you would want, honestly. We have **structured request logging tagged with the effective user id** on every call, and a `CreatedBy` field on posts, but there is **no dedicated audit-log service** recording token use action by action. So you end up reconstructing activity from the request logs and the posts that showed up, rather than reading a purpose-built audit trail. Naming that gap is the honest answer.

13. AI generation is rate-limited per business, and partners buy different tiers. Is that authorization, quota, or entitlement, and where does the check belong?

They are actually **three different things**, and we handle them in three different places. **Authorization**, the IAM `AccessResource` check, decides whether you may touch this business at all. **Quota** is a per-business rate limit, a Redis sliding window, checked in the handler after authorization. And **entitlement**, which tier the partner bought, is not a live per-request check at all, it is applied **once at provisioning time** when a marketplace edition is activated.

How do you gate a premium feature per partner in a white-label product?

Through the **marketplace edition**. When a business gets provisioned onto a premium edition, a workflow switches on the premium capability, the AI Employees, keyed off the edition id. It is a **one-time activation**, not a flag re-checked on every generation call, so "premium" just means the capability got turned on, not that each request re-verifies the tier.

Where does an entitlement check sit relative to the authz check in the request path?

On the paths that have both, the order is **authorization first, then quota**, both before the expensive model call. Entitlement is **out of band**, decided at provisioning rather than in the request path, so a generation request does not re-ask “is this partner premium”, it just finds the capability there or not.

What stops a client from calling the gRPC directly and skipping the UI gate?

The **server-side IAM check in the handler**, not the UI. The UI gate is cosmetic; the real boundary is the `AccessResource` call that each RPC makes.

> **Honest gap**
>
> One legacy image endpoint (`CreateImages`) skips the IAM check entirely, with a comment that it was dropped to unblock the email builder. On that path, calling the gRPC directly does bypass authorization. The fix is to enforce IAM on every RPC and never rely on the UI.

14. Using Vendasta’s IAM, how would you model the permission that decides whether a user can publish for a given business?

I would model it against the shared **AccountGroup resource** in IAM, checked with `AccessResource(caller, accountGroup, ActionWrite)` . The account group is the business, so “can publish for this business” becomes “may this caller take the write action on this account group”, and IAM evaluates that centrally against the caller’s claims.

RBAC or ABAC for this, and what pushes you one way?

It is **role-scoped ABAC**. The role (Partner, SMB, DigitalAgent, Superadmin, service account) picks which rule applies, and the rule itself is **attribute matching**, comparing the caller’s `partner_id` and group associations against the resource’s `account_group_id` and `partner_id` . Pure RBAC just cannot express “this partner admin, but only for businesses under their own partner”, which is the multi-tenant requirement, so the attributes are unavoidable.

Where do you register the resource and action, and where does the policy live?

In the **iam-resources repo**, one package per owning service (for example `applications/social-posts`). The resource declares its attributes and an `AccessRules` map of action to role to policy. The policy itself gets evaluated at request time by the **central IAM service**, so the repo holds the declaration and IAM holds the engine.

How do you keep the check consistent across social-posts, social-drafts, and CS instead of three drifting copies?

By not copying it. The business-level gate is the single AccountGroup resource owned by account-group-microservice, and all three call it, social-posts and CS through their own language SDKs, all referencing the same definition. The finer resources (MultiLocationPost, Draft) are each declared once by their owning service and imported, so there is one definition per permission, not one per service.

15. The network access tokens are secrets. Where do they live, how are they protected at rest, and what is the blast radius if that store leaks?

They live **right next to the service that uses them**: Facebook and most networks in **CS's App Engine Datastore**, Instagram in the instagram service's **VStore**. The honest finding is that at the application level they are stored as **plain strings**, no field-level encryption, no KMS envelope, no column encryption in the read or write path. So the at-rest protection really rests on **GCP's platform default encryption** of the underlying stores, not on anything the code adds. And the blast radius, if a store leaked past that platform encryption, is **every connected account in that store**.

> **Honest finding**
>
> There is no application-managed encryption on these token fields, so there is no envelope key to rotate. Adding KMS envelope encryption (a per-record data key wrapped by a KMS key) would give a real at-rest boundary and a key you could rotate. That is what I would add.

GCP Secret Manager, KMS envelope encryption, or database column encryption? Which and why?

None of those wrap the tokens today. Secret Manager holds the OpenAI key, and KMS is used by another service (sso), but the network tokens sit in the datastores as plaintext fields. If I were designing it fresh, **KMS envelope encryption** fits best: these tokens are high-value, machine-read secrets, and envelope encryption gives you per-record data keys and a rotatable master key without shoving millions of per-business rows into Secret Manager, which is not built for that shape.

Who, human or service, can actually read a raw token?

Any service that passes the IAM resource check on the owning service's RPC. And worth calling out, the Instagram user record ships the `access_token` field **right in its proto response**, so a caller allowed to read the user reads the token too, with no field-level redaction. That is broader than ideal, and tightening it (redact the token, expose a use-the-token RPC instead of a read-the-token RPC) is a real improvement.

How would you rotate the encryption key without downtime?

There is no application key to rotate today, which is itself the gap. With KMS envelope encryption in place, you rotate the master key by **re-wrapping the per-record data keys** in the background: keep the old key version active for decryption, encrypt new writes under the new version, re-encrypt the existing records lazily or in a sweep, then retire the old version once nothing references it. No request path has to stop.

16. A Temporal workflow wakes days after it was created and publishes. There is no live user session at that moment. Under whose authority does it act, and how does it authenticate to CS and the network?

It acts under **social-posts' own service identity**, not the user's, the user's session is long gone by then. When the workflow wakes and calls CS, social-posts signs a short-lived JWT with its own **client key**, exchanges it with IAM for a **service session token**, and calls CS with that. Then CS makes the actual network call using the **stored network OAuth token** for that business. So there are two credentials in play, and neither is the user's: the service token for the internal hop, and the network token for the external post.

Was authorization checked at schedule time or publish time, and does that gap matter?

At **schedule time**, in the handler that created the workflow. There is **no re-check at publish time**. And yes, the gap matters, because anything that changes the user's authority between scheduling and firing is not reconsidered.

> **The gap**
>
> If the user lost permission or was deactivated after scheduling, the post still fires, because the workflow runs under the service identity and authorization was already decided at schedule time. A publish-time re-check, or cancelling a user's scheduled posts on deactivation, would close it.

What if the user lost permission, or was deactivated, in between?

As I said, the post publishes anyway today. Closing that means either **re-authorizing at publish time** against the current claims, or **cancelling the workflows tied to a user** when they are deactivated. The first is cleaner but it adds a check to every publish; the second is an event-driven cleanup.

How does a long-lived background job hold a valid credential without a user present?

It does not hold a long-lived user token at all. It holds the service's own **client key** (a Kubernetes secret) and mints a **fresh short-lived session token per call** through the IAM exchange. So the credential is always freshly minted, never a stale user session, which is exactly why a workflow can wake up two weeks later and still authenticate cleanly.